System Architecture and Integration

TECHNICAL DOCUMENTATION

Containerized Infrastructure and CI/CD Automation using Docker and Jenkins

Project: **EcomApp** - a buyer/seller online marketplace

Item	Details
Group name	[enter group name]
Course / Section	[enter course and section]
Repository	https://github.com/JayIOIO/FinalProject-CI-CD-Automation
Instructor	[enter instructor name]
Submission date	[enter date]

Group members and roles

Member	Primary role	Main outputs
Salenga	Project Lead / Scrum Master	Proposal, task board, contribution log, integration coordination, presentation flow
Riboroso	DevOps / CI-CD Engineer; Backend and Database Engineer*	Jenkins in Docker, Jenkinsfile, trigger, credentials; backend tests, SQL schema, data contracts
Tato	Infrastructure Engineer	Architecture, Dockerfiles, docker-compose.yml, NGINX proxy, network, volume, health checks
Ash	Frontend, QA and Documentation Lead	UI validation, smoke / integration tests, README, this document, slides
Fifth member	[name, GitHub account, assigned role - to be confirmed]	[assigned deliverables]

* Riboroso currently holds two roles because only four distinct members have been named. Once the fifth member is confirmed, one role should be transferred and this table, the proposal, README, contribution log and slides updated together.

How to read this document. Text on a yellow background marks a detail that must be confirmed against the repository, or evidence (screenshot, run ID) that the team must capture. Per the project guide, no trigger, quality-gate, persistence or rollback proof is claimed until it has been performed and captured. Code excerpts show the design agreed in the team guide; replace them with the exact files from the repository before submission.

1. Project Description and Modules

EcomApp is a small online marketplace in which **sellers** publish and manage product listings and **buyers** browse, check out and track orders. The assignment grades the infrastructure and automation rather than the feature count, so the application is intentionally small: its purpose is to be the payload that Docker packages and Jenkins delivers automatically.

Starting point. The team started from a feature-stage baseline (`ecomapp-feature-base`) that already contained the React frontend, three Node.js API services and the MySQL schema. These application features are part of the supplied baseline and are not claimed as the team's original work. The team's own contribution is the validation of those features, the automated tests, the Docker and Compose layer, the reverse proxy, Jenkins, the Jenkinsfile, the proofs of the quality gate, persistence and rollback, and this documentation.

1.1 Module responsibilities

Module (container)	Source folder	Responsibility	Health
frontend	frontend/	React (Vite) buyer and seller screens: browse and search, sign up / log in, seller listings, checkout, order history. Shows a visible build/version label used to prove a deployment.	static page
auth-service	services/auth-service/	Buyer and seller signup and login, password hashing, JWT issuing, profile updates.	GET /health
catalog-service	services/catalog-service/	Public product search and seller-owned inventory endpoints (create, update, list own products).	GET /health
order-service	services/order-service/	Buyer checkout and order history; seller fulfillment operations. Verifies JWTs signed by auth-service.	GET /health
db	db/init.sql	MySQL 8: users, products, orders, order_items, relationships and initial sample products. Data lives in a named volume.	mysqladmin ping
proxy	proxy/	NGINX: the single public entry point; routes / and /api/* to the correct container.	HTTP 200 on /
jenkins (separate)	infra/jenkins/	CI/CD server with Docker CLI; builds, tests and deploys the application stack.	UI on :8081

The system satisfies the minimum of three application modules (auth, catalog, order, plus the frontend), each in its own container. The auth, catalog and order services all read from and write to the database, and each backend exposes a health endpoint used by Docker health checks and by the pipeline's smoke test.

1.2 Primary user journeys

- **Seller:** sign up as seller, log in, create a product listing, see it in own inventory, fulfil an incoming order.
- **Buyer:** sign up as buyer, search the catalog, add items and check out, view order history.
- **Authorization:** a buyer cannot call seller endpoints; a user sees only their own orders and inventory.

2. Architecture

Figure 1 shows the final target system: the application stack and the Jenkins stack run as two independent Compose projects on one Docker host. Only the NGINX proxy publishes a host port for the application; the database and all APIs are reachable only on the private network.

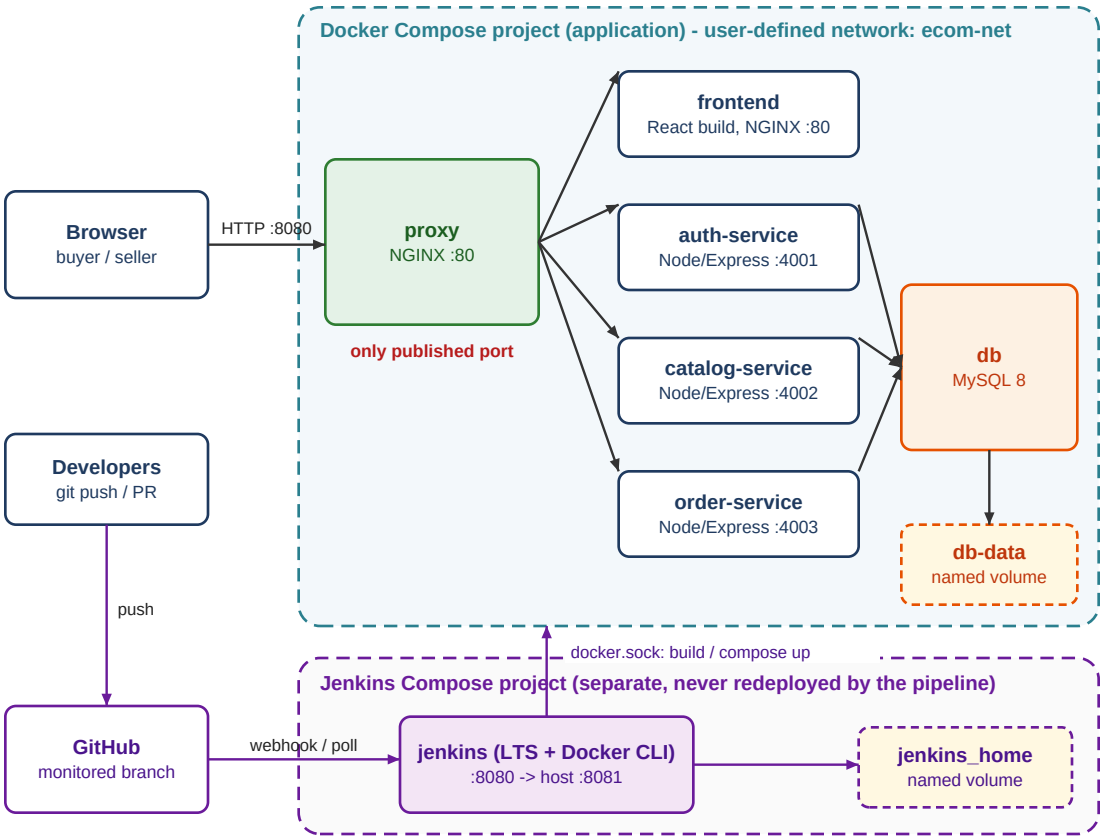

Figure 1 - Architecture: containers, ports, network, volumes and CI/CD flow. [verify host port and names against docker-compose.yml]

2.1 Request flow

- The browser calls the proxy on a single host port (for example `http://localhost:8080`).
- `/` goes to the **frontend**; `/api/auth/` to **auth-service**; `/api/products/` to **catalog-service**; `/api/orders/` to **order-service**.
- Inside the network, services reach each other by DNS name (`db`, `auth-service`, ...). The same code that connects to `127.0.0.1` in local development uses `db` in Compose.
- Protected calls carry a JWT issued by auth-service; catalog and order services verify it with the same signing secret.

2.2 Container, port and network summary

Container	Internal port	Published to host	Network	Volume
proxy	80	**Yes** (single entry point)	ecom-net	-
frontend	80	No	ecom-net	-
auth-service	4001	No	ecom-net	-
catalog-service	4002	No	ecom-net	-
order-service	4003	No	ecom-net	-
db (MySQL)	3306	No	ecom-net	db-data (named)

Container	Internal port	Published to host	Network	Volume
jenkins	8080 (UI), 50000	Yes (8081, lab only)	own Compose network	jenkins_home (named)

Design rationale. A single published port reduces the attack surface and gives the browser one origin, which avoids CORS problems. Keeping Jenkins in a separate Compose project means a pipeline run that redeploys the application can never restart the CI server that is executing it.

3. Technology Stack and Justification

Component	Choice	Why this was chosen
Frontend	React + Vite	Supplied with the baseline; fast build; produces static files that NGINX can serve with a tiny runtime image.
Backend	Node.js 20 + Express (three services)	Supplied with the baseline; one language across the team; small Alpine images; simple `npm test` and `npm ci` workflows.
Database	MySQL 8	Relational data (users, products, orders, order items with foreign keys); official image supports init scripts and a named volume.
Reverse proxy	NGINX (pinned Alpine tag)	Lightweight, well documented path-based routing; single public entry point.
Containers	Docker + Docker Compose	Reproducible images and one-command startup of the whole stack.
CI/CD server	Jenkins LTS (JDK 17) in Docker	Required by the course; pipeline as code through a declarative Jenkinsfile; persistent jenkins_home.
Git hosting	GitHub	Pull-request review workflow; push-event webhook support.
Pipeline trigger	[GitHub webhook via tunnel OR Poll SCM]	[one sentence: why this trigger suits the lab / demo network]
Testing	[confirm: e.g. Node test runner / Jest + curl smoke test]	Tests run inside Docker (multi-stage `test` target) so the Jenkins host needs no Node installation.

4. Dockerfile Explanations

Every module the team builds has its own Dockerfile and `.dockerignore`. The official MySQL image is used for the database with the SQL schema mounted or copied as an init script. All base images are pinned to a specific version; `latest` is not used.

4.1 Backend services (auth, catalog, order)

```
# Representative excerpt - confirm against services/<name>/Dockerfile
FROM node:20-alpine AS base
WORKDIR /app
COPY package*.json ./            # dependency manifests first: cached layer
RUN npm ci
COPY . .

FROM base AS test                # built by Jenkins: docker build --target test
RUN npm test                     # a failing test fails the image build

FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev            # no dev dependencies in the runtime image
COPY --from=base /app ./
EXPOSE 4001
HEALTHCHECK --interval=15s --timeout=3s CMD wget -qO- http://localhost:4001/health || exit 1
USER node                        # non-root runtime user
CMD ["node", "server.js"]
```

4.2 Frontend and proxy

```
# frontend/Dockerfile (representative)
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG BUILD_NUMBER=dev
ENV VITE_BUILD_NUMBER=$BUILD_NUMBER              # shown in page footer
RUN npm run build
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

```
# proxy/Dockerfile (representative)
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

4.3 Design choices

Practice	How it is applied	Benefit
Pinned slim base images	node:20-alpine, nginx:1.27-alpine, mysql:8.x	Reproducible builds, small image size, fewer vulnerabilities.
Layer caching	package*.json copied and installed before source code	Code-only changes rebuild in seconds because the dependency layer is reused.
Multi-stage with test stage	base -> test -> production; frontend build -> NGINX	Tests run in Docker; dev tools and build chain do not ship in the runtime image.
Non-root user	USER node in service images	A compromised process has no root privileges in the container.
HEALTHCHECK	wget to /health in each backend	Compose and the smoke test can tell healthy from merely running.
Config baked in	nginx.conf is COPYed into the proxy image (no bind mount)	Works the same from Jenkins' workspace as on a developer laptop.
.dockerignore	node_modules, .git, .env, dist excluded	Smaller build context and no secrets leaked into images.
Build-number argument	BUILD_NUMBER passed to the frontend build	A visible version label proves which build is live.

5. Docker Compose Explanation

One command, `docker compose up -d --build`, builds and starts the complete application. Jenkins is deliberately kept out of this file.

```
# Representative excerpt - confirm names against docker-compose.yml
services:
  proxy:
    build: ./proxy
    image: ecomapp/proxy:${TAG:-latest}
    ports: ["8080:80"]                      # the ONLY published port
    depends_on: [frontend, auth-service, catalog-service, order-service]
    restart: unless-stopped
    networks: [ecom-net]
  auth-service:
    build: { context: ./services/auth-service, target: production }
    image: ecomapp/auth-service:${TAG:-latest}
    environment:
      DB_HOST: db
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      db: { condition: service_healthy }    # start only when MySQL answers
    restart: unless-stopped
    networks: [ecom-net]
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
    volumes:
      - db-data:/var/lib/mysql              # named volume = persistence
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks: [ecom-net]
volumes: { db-data: {} }
networks: { ecom-net: {} }
```

Concept	Configuration in EcomApp	Why it matters
depends_on + health	APIs wait for db with condition: service_healthy; the proxy waits for the services.	Prevents start-up race conditions where an API crashes because MySQL is still initialising.
Health checks	db (mysqladmin ping) and each backend (/health).	Distinguishes a healthy service from a running but broken container.
User-defined network	ecom-net; service names act as DNS names.	Isolation from other containers; no hard-coded IP addresses.
Named volume	db-data mounted at /var/lib/mysql.	Data survives container removal, redeploys and docker compose down (without -v).
Environment variables	Passwords, DB settings and JWT secret come from .env (local) or Jenkins Credentials (pipeline). .env is git-ignored; .env.example holds placeholders.	No secrets in Git or in images.
Restart policy	restart: unless-stopped.	Services recover automatically after a crash or host reboot.
Image tags	image: ecomapp/<module>:${TAG:-latest}; Jenkins sets TAG to the build number.	Every build is a distinct, redeployable image: the basis of rollback.
Published ports	Only the proxy publishes a port; db and APIs do not.	Database is never exposed to the host or network.

6. Jenkins in Docker

6.1 Installation

Jenkins lives in `infra/jenkins/` with its own Dockerfile and `docker-compose.yml`, separate from the application stack. The image extends `jenkins/jenkins:lts-jdk17` and adds the Docker CLI and the Compose plugin so pipeline steps can run `docker build` and `docker compose` against the host daemon.

```
# infra/jenkins/docker-compose.yml (representative)
services:
  jenkins:
    build: .
    container_name: jenkins
    user: root                      # lab only: needed to use the Docker socket
    restart: unless-stopped
    ports: ["8081:8080", "50000:50000"]
    volumes:
      - jenkins_home:/var/jenkins_home          # persistent configuration + history
      - /var/run/docker.sock:/var/run/docker.sock
volumes:
  jenkins_home:
```

Step	Command / action
Start Jenkins	`docker compose -f infra/jenkins/docker-compose.yml up -d --build`
Unlock	`docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`, then open http://localhost:8081
Plugins	Suggested plugins plus **Docker Pipeline**; [add GitHub / other plugins actually installed]
Verify Docker access	`docker exec jenkins docker ps` lists the host's containers
Credentials	Manage Jenkins > Credentials: secret text for DB password, MySQL root password, JWT secret (IDs: `ecommerce-db-password`, `ecommerce-mysql-root-password`, `ecommerce-jwt-secret`) [confirm IDs]; Git token if the repo is private
Job	Pipeline > *Pipeline script from SCM*; repository URL, monitored branch [main / other], Script Path = `Jenkinsfile`

6.2 Automatic trigger

The pipeline must start from a push, not from **Build Now**. The team uses [Option A: GitHub webhook / Option B: Poll SCM]:

- **Webhook (preferred):** job option *GitHub hook trigger for GITScm polling*; GitHub payload URL `https://<public-jenkins-url>/github-webhook/`, content type JSON, push events. Because Jenkins runs on a laptop, a tunnel such as ngrok exposes port 8081.
- **Poll SCM (fallback):** schedule `H/2 * * * *` (every ~2 minutes); needs no public URL, but reaction can take up to the polling interval.

The first build after setup is run manually only as setup validation; it is not counted as proof of automation.

INSERT SCREENSHOT: Jenkins job configuration / GitHub webhook delivery

Show trigger setting and a successful (green) webhook delivery, or the Poll SCM schedule. Redact tokens.

6.3 Security note

Mounting `/var/run/docker.sock` and running as root gives Jenkins full control of the host's Docker daemon. This is acceptable in a classroom lab but not in production. Safer alternatives are dedicated build

agents, rootless Docker, Docker-in-Docker with TLS, or daemonless builders such as Kaniko.

7. Pipeline Stages and Failure Behavior

The declarative `Jenkinsfile` sits in the repository root (pipeline as code). The pipeline order is **Checkout > Test > Build Images > Deploy > Smoke Test / Verify**; an integration-test stage may run before Deploy.

```
// Representative structure - confirm against the repository's Jenkinsfile
pipeline {
  agent any
  environment { COMPOSE_PROJECT_NAME = 'ecomapp'; TAG = "${env.BUILD_NUMBER}" }
  triggers   { githubPush() }                // or pollSCM('H/2 * * * *')
  options    { disableConcurrentBuilds(); buildDiscarder(logRotator(numToKeepStr: '15')) }
  stages {
    stage('Checkout')     { steps { checkout scm } }
    stage('Test')         { steps {
        sh 'docker build --target test -t ecomapp/auth-service:test ./services/auth-service'
        sh 'docker build --target test -t ecomapp/catalog-service:test ./services/catalog-service'
        sh 'docker build --target test -t ecomapp/order-service:test ./services/order-service' } }
    stage('Build Images') { steps { sh 'docker compose build' } }
    stage('Deploy')       { steps {
        withCredentials([string(credentialsId: 'ecommerce-db-password', variable: 'DB_PASSWORD'),
                         string(credentialsId: 'ecommerce-jwt-secret',  variable: 'JWT_SECRET')]) {
          sh 'docker compose up -d --no-build --remove-orphans' } } }
    stage('Smoke Test')   { steps { sh './scripts/smoke-test.sh' } }   // via the proxy
  }
  post {
    success { echo "EcomApp build ${TAG} is live" }
    failure { echo "Build ${TAG} failed. Roll back: TAG=<previous> docker compose up -d --no-build" }
    always  { sh 'rm -f .env' }
  }
}
```

Stage	What it does	On failure
Checkout	Fetches the pushed commit from the monitored branch.	Pipeline stops; nothing is built or deployed.
Test	Builds each backend's *test* target; `npm test` runs inside Docker.	**Quality gate:** the build fails here and Deploy never runs; the previous version keeps serving users.
Build Images	Builds production images tagged with the build number.	Stops before Deploy; running containers are untouched.
Deploy	Injects secrets from Jenkins Credentials and runs `docker compose up -d --no-build`.	Compose keeps healthy containers; post section prints the rollback command.
Smoke Test / Verify	Polls health endpoints and the home page through the proxy; retries while services warm up.	Build marked failed; operator rolls back to the previous tag.
post	Prints success / failure with build tag; always deletes the temporary .env.	Secrets never remain in the workspace.

INSERT SCREENSHOT: Jenkins Stage View of a successful run

Capture the real Stage View with build number; all stages green.

Run reference: [build # / commit hash / date of the successful automatic run]

8. Testing Strategy

The strategy has three layers: (1) **unit/API tests** for each backend service, run in the Dockerfile *test* stage; (2) an **integration test** that exercises the buyer and seller journeys through the proxy against a disposable test stack and database (never the persistent demo database); and (3) a **smoke test** after every deployment.

8.1 Test cases

Service	Case	Expected result
All backends	GET /health	HTTP 200, no secrets in body
auth-service	Sign up buyer and seller with valid data	Account created; password not returned
auth-service	Sign up with an existing email / missing field	Rejected with 4xx
auth-service	Log in with correct / wrong password	JWT issued / 401
auth-service	Update profile without / with token	401 / success
catalog-service	Public product search	List returned without authentication
catalog-service	Seller creates and edits own product	Saved in DB; visible in own inventory
catalog-service	Buyer calls seller inventory endpoint	403 / 401
order-service	Buyer checks out valid cart	Order and order items written; total correct
order-service	Buyer views history	Only the buyer's own orders
order-service	Seller updates fulfillment status	Status changes; other sellers' orders blocked
Smoke (proxy)	GET / and /api/*/health through the proxy	All HTTP 200 after deploy

[replace with the exact test file names and counts from the repository, e.g. tests/ folders and framework]

8.2 Proof that a failing test blocks deployment

Procedure (as required by the course): start from a passing deployed build and record its tag and visible build label; push a commit containing a deliberate test failure to the monitored branch; let the automatic trigger start the pipeline; show that it fails at **Test** and that **Deploy** is skipped; confirm the browser still shows the previous build label; then revert the commit through a reviewed change and show the pipeline passing again.

INSERT SCREENSHOT: Failing run: Stage View with Test failed and Deploy skipped

Capture the red Test stage and grey/skipped later stages.

INSERT SCREENSHOT: App still on previous version + recovery run green

Browser footer showing the earlier build, then the later passing run.

Run references: failing build [#], previous good build [#], recovery build [#]

9. Rollback Procedure and Data Persistence

9.1 Data persistence

MySQL stores its files in the named volume `db-data`. Replacing or removing application containers does not remove the volume, so records survive redeploys.

#	Action	Command
1	Create a safe, recognizable record (e.g. a seller listing or order)	via the UI through the proxy
2	Confirm the volume exists and note the project name	`docker volume ls`
3	Stop and restart without deleting volumes	`docker compose down` then `docker compose up -d`
4	Check all services are healthy	`docker compose ps`
5	Show the same record is still present	UI or SQL query

Warning: `docker compose down -v` deletes named volumes and must never be run on the shared or presentation stack.

INSERT SCREENSHOT: Record before and after redeploy + docker volume ls

Same record visible in both screenshots; timestamps visible; redact passwords.

9.2 Rollback to an earlier image tag

Because every deployed image is tagged with its build number, an earlier version can be redeployed without rebuilding:

```
# 1. list available tags / confirm two successful builds exist on the host
docker images "ecomapp/*"
# 2. redeploy the known-good build (example: 15) into the same Compose project
TAG=15 docker compose -p ecomapp up -d --no-build
# 3. verify
docker compose -p ecomapp ps
curl -fsS http://localhost:8080/api/auth/health
```

Item	Rolled back?	Notes
Application images (frontend, 3 APIs, proxy)	**Yes**	Restored to the chosen TAG; the visible build label returns to that number.
MySQL data and schema	**No**	The volume is unchanged; a schema change would need a separate migration or restore strategy.
Jenkins configuration	No	Separate Compose project and volume; unaffected.

INSERT SCREENSHOT: Rollback: command output and browser showing earlier build label

Include current build, target build, and healthy `docker compose ps`.

Rollback reference: from build [#] to build [#]

10. Problems Encountered and Solutions

This section must list problems the group **actually** met. The rows below are placeholders; keep only those that occurred and add the rest from the team's issue board and commit history.

Problem / symptom	Cause	Solution	Owner
[e.g. API could not reach MySQL inside Compose]	[host was 127.0.0.1 instead of service name db]	[DB_HOST=db via Compose environment]	[name]
[e.g. API crashed on first start]	[MySQL not ready; no health-based depends_on]	[added healthcheck + condition: service_healthy]	[name]
[e.g. Jenkins: docker: command not found / permission denied]	[no Docker CLI or socket access]	[custom Jenkins image + socket mount]	[name]
[e.g. webhook never fired]	[Jenkins not publicly reachable]	[ngrok tunnel / switched to Poll SCM]	[name]
[e.g. pipeline ignored pushes]	[branch condition did not match default branch]	[aligned Jenkins job and Jenkinsfile branch]	[name]
[add row]			
[add row]			

Known limitations (disclose, do not hide)

- Jenkins uses the host Docker socket and root: lab-only, not production-grade (see 6.3).
- Rollback restores application images only, not database contents or schema.
- Everything runs on a single Docker host; there is no registry, high availability or staging environment [edit if bonus features were built].

11. Individual Contributions

Contributions are supported by GitHub commits, pull requests and issue-board cards. Every member has commits in the repository and no one merged their own pull request without independent review.

Member	Role	Contributions (issues / PRs / evidence)	Presentation segment
Salenga	Project Lead / Scrum Master	[proposal, board, contribution log, PR coordination, PR numbers]	[]
Riboroso	DevOps / CI-CD; Backend and DB	[Jenkins setup, Jenkinsfile, trigger, credentials, tests, schema; PR numbers]	[]
Tato	Infrastructure Engineer	[Dockerfiles, Compose, proxy, network, volume, health checks; PR numbers]	[]
Ash	Frontend, QA and Documentation	[UI version label, smoke / integration tests, README, this PDF, slides; PR numbers]	[]
Fifth member	[role]	[name and contributions]	[]

12. References

- System Architecture and Integration - Final Project Guidelines: Containerized Infrastructure and CI/CD Automation using Docker and Jenkins (course handout).
- Docker Documentation. Dockerfile reference, multi-stage builds and best practices. https://docs.docker.com/
- Docker Documentation. Docker Compose file reference (healthcheck, depends_on, volumes, networks). https://docs.docker.com/compose/

- Jenkins Documentation. Pipeline syntax, Jenkinsfile and Using Docker with Pipeline. https://www.jenkins.io/doc/book/pipeline/
- Jenkins Documentation. Installing Jenkins with Docker. https://www.jenkins.io/doc/book/installing/docker/
- GitHub Docs. Webhooks and pull requests. https://docs.github.com/
- NGINX Documentation. Reverse proxy and proxy_pass. https://nginx.org/en/docs/
- MySQL 8 Reference Manual and the official MySQL Docker image. https://hub.docker.com/_/mysql
- Project repository and team execution guide (EcomApp). https://github.com/JayIOIO/FinalProject-CI-CD-Automation